Scalability Report for the Tensorial Risk Engine

April 15, 2026

Objective

This note summarizes a scalability experiment for the Tensorial Risk Engine focused on: (i) CPU-versus-GPU comparisons, (ii) separate timings for forward and backward passes, (iii) scaling studies with increasing numbers of assets and events, and (iv) a concise discussion of memory requirements.

The experiment is reproducible with the script `software_x_scalability.py`, which generates the benchmark tables, summary file, and figures used in this report.

The benchmark was executed with the manuscript implementation of the `TensorialRiskEngine`. Two parameter sweeps were performed:

- **Asset scaling:** number of assets increased from 100 to 10,000 while keeping the number of events fixed at 2,000.
- **Event scaling:** number of events increased from 100 to 20,000 while keeping the number of assets fixed at 1,000.

For each case, the script recorded:

- forward-only execution time,
- backward time with respect to the vulnerability matrix,
- full forward-plus-backward execution time for the gradients with respect to hazard, vulnerability, exposure, and event rates.

Main Findings

The results show a clear split between forward-only evaluation and differentiable analysis:

- For **forward-only** execution, the CPU is faster than the GPU over the full tested range. This suggests that, at these sizes, GPU launch and data-movement overhead dominate the benefit of parallel execution.
- For the **full differentiable workflow**, the GPU becomes advantageous once the problem is sufficiently large. The crossover is visible around $N \approx 1000$ assets for the asset-scaling test and between $Q = 1000$ and $Q = 5000$ events for the event-scaling test.
- At the largest tested cases, the GPU provides approximately a **2x speedup** for full forward-plus-backward execution:
  - $N = 10{,}000$, $Q = 2{,}000$: CPU 0.2287 s, GPU 0.1116 s, speedup $\approx 2.05\times$.

- $N = 1{,}000$, $Q = 20{,}000$: CPU 0.2259 s, GPU 0.1148 s, speedup $\approx 1.97\times$.

- The backward cost grows in a manner consistent with the increase in tensor dimensions. This is expected because the full gradient analysis must retain and traverse the same graph that produces the probabilistic loss matrix.

Memory Discussion

The dominant memory terms scale as $\mathcal{O}(NQ)$, where N is the number of assets and Q is the number of events. In single precision (`float32`), the main tensors require approximately:

- hazard matrix $H \in \mathbb{R}^{N\times Q}$: $4NQ$ bytes,
- loss matrix $J \in \mathbb{R}^{N\times Q}$: $4NQ$ bytes,
- hazard gradient $\partial\mathrm{AAL}/\partial H \in \mathbb{R}^{N\times Q}$: $4NQ$ bytes.

Therefore, the full backward pass is more memory intensive than forward-only evaluation, because the autodiff engine must retain intermediate tensors and, when requested, materialize gradients of the same order. In the present benchmark, the estimated working memory for the largest tested cases was approximately 228.9700 MB.

Representative Numerical Results

Table 1: Asset-scaling study with $Q = 2000$ fixed.

Device	N	Forward [s]	Full F+B [s]	Memory [MB]	GPU speedup
CPU	100	0.0138	0.0055	2.30	—
GPU	100	0.0542	0.0145	2.30	0.38
CPU	1000	0.0216	0.0239	22.90	—
GPU	1000	0.0630	0.0220	22.90	1.09
CPU	5000	0.0500	0.1128	114.49	—
GPU	5000	0.0797	0.0590	114.49	1.91
CPU	10000	0.0854	0.2287	228.97	—
GPU	10000	0.0923	0.1116	228.97	2.05

Table 2: Event-scaling study with $N = 1000$ fixed.

Device	Q	Forward [s]	Full F+B [s]	Memory [MB]	GPU speedup
CPU	100	0.0137	0.0041	1.15	—
GPU	100	0.0589	0.0114	1.15	0.36
CPU	1000	0.0182	0.0143	11.46	—
GPU	1000	0.0619	0.0165	11.46	0.87
CPU	5000	0.0347	0.0576	57.25	—
GPU	5000	0.0659	0.0336	57.25	1.72
CPU	10000	0.0495	0.1112	114.49	—
GPU	10000	0.0771	0.0614	114.49	1.81
CPU	20000	0.0863	0.2259	228.97	—
GPU	20000	0.0992	0.1148	228.97	1.97

Figures

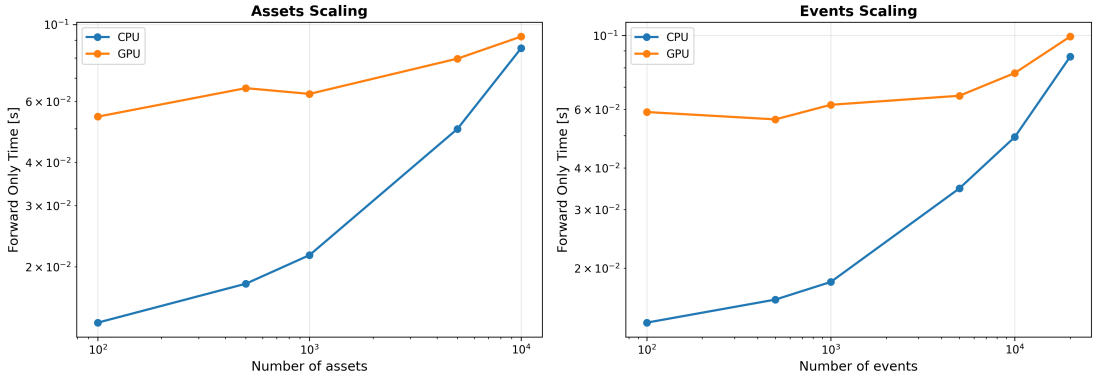

Figure 1: Forward-only timing as a function of assets and events. In the tested range, the CPU remains faster than the GPU for the pure forward evaluation.

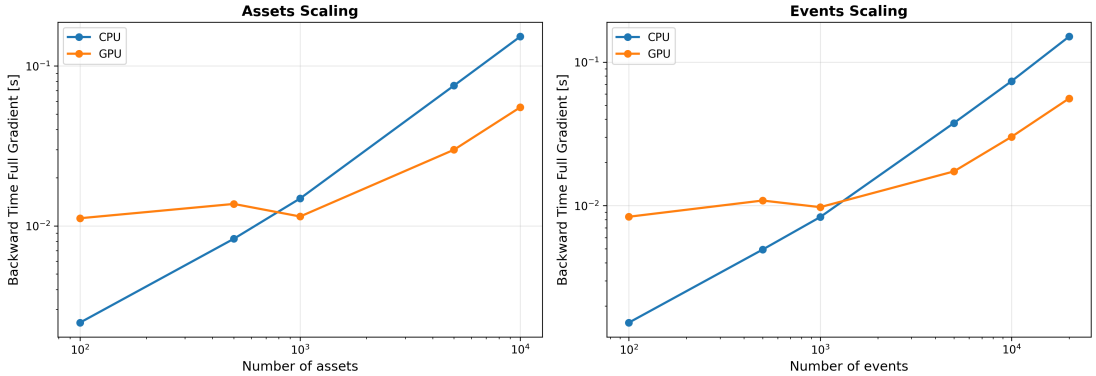

Figure 2: Backward timing for the full gradient analysis. The GPU becomes advantageous as the problem size increases.

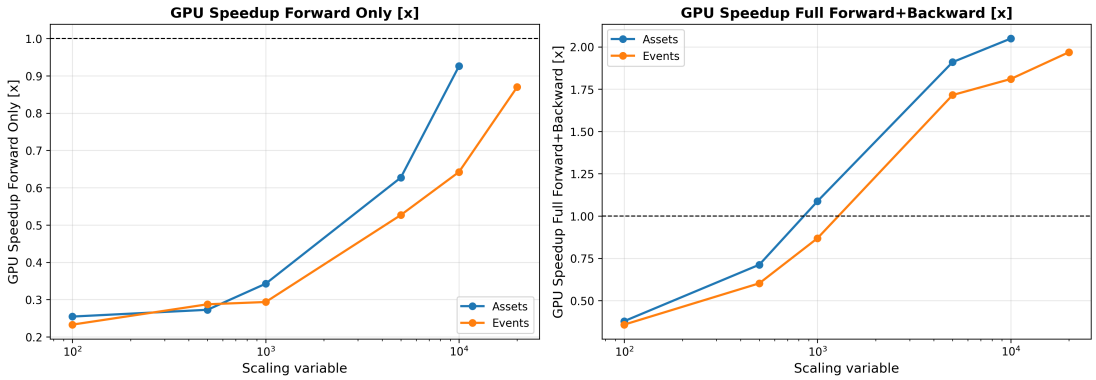

Figure 3: GPU speedup relative to the CPU. Speedup below unity indicates that the GPU is slower; speedup above unity indicates a net GPU advantage.

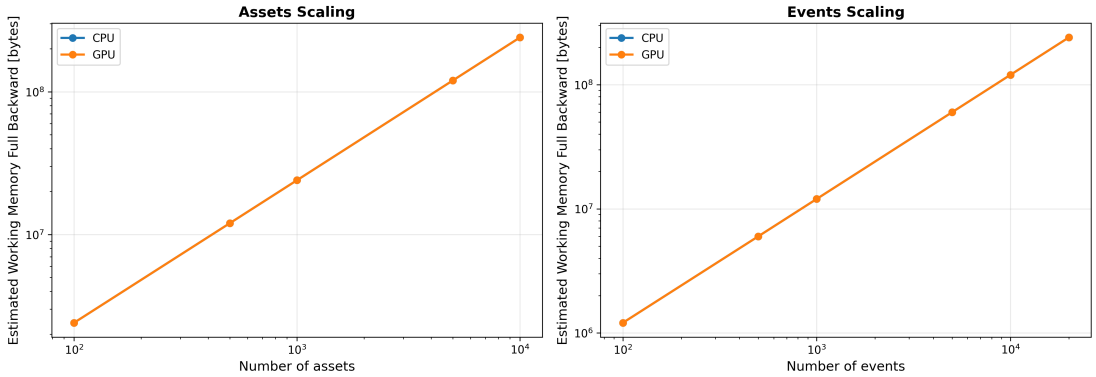

Figure 4: Estimated working memory for the full backward pass. The dominant trend follows the expected $\mathcal{O}(NQ)$ scaling.

Conclusion

The benchmark shows that the tensorial engine scales predictably with the number of asset-event combinations and that the memory footprint is governed by the probabilistic loss tensor and its gradients. The most important practical result is that GPU acceleration is not uniformly beneficial for small forward-only workloads, but it becomes clearly advantageous for large differentiable analyses, where the full forward-plus-backward workflow reaches roughly a twofold speedup at the largest tested problem sizes.